

■ Identificação do Grupo — TechVerse

RM	Nome Completo
566223	Davi Marques de Andrade Munhoz
562559	Diogo Oliveira Lima
566539	Leandro Simoneli da Silva
562414	Lucas dos Reis Aquino
565356	Lucas Perez Bonato

1. Contextualização e Justificativa do Cenário

O Brasil é um dos países mais vulneráveis ao impacto das mudanças climáticas. As enchentes históricas no Rio Grande do Sul em maio de 2024 afetaram 478 municípios, deslocaram mais de 500 mil pessoas e causaram prejuízos superiores a R\$ 60 bilhões. Paralelamente, a região do MATOPIBA (Maranhão, Tocantins, Piauí e Bahia) concentra municípios com crescente risco de seca, impulsionado pelo avanço do agronegócio e pela redução das chuvas. Satélites como o GOES-16 e a constelação Sentinel transmitem dados em tempo real sobre temperatura, precipitação e cobertura vegetal — gerando redes complexas de informação geoespacial que precisam ser organizadas, percorridas e analisadas com eficiência.

Este projeto implementa um sistema computacional de monitoramento e triagem de riscos ambientais, traduzindo o ecossistema da economia espacial em um problema concreto de grafos e estruturas de dados. Foram selecionados dois cenários: **Cenário A** — rede de resposta a enchentes no RS (N=12 municípios no grafo de trabalho, 80 nos dados brutos) — e **Cenário B** — triagem de risco de seca no MATOPIBA (N=10 municípios no grafo de trabalho, 50 nos dados brutos). A escolha é justificada pela disponibilidade de dados, relevância social e alinhamento aos ODS 2, 9, 11 e 13 da ONU.

2. Modelagem: Grafo, BST e Estruturas de Dados

2.1 Grafo de Municípios

O grafo $G = (V, E)$ é representado como **dicionário de listas de adjacência**: $adj = \{id: [(vizinho, peso), ...]\}$. Cada vértice é uma **tupla imutável** ($id_municipio, nome, indice_risco, custo_atendimento, populacao$). O peso das arestas representa o tempo de deslocamento em horas (Cenário A) ou a distância rodoviária em km (Cenário B). O Cenário A possui 12 vértices e 15 arestas; o Cenário B, 10 vértices e 14 arestas.

Justificativa da lista de adjacência frente à matriz: para grafos esparsos — municípios tipicamente têm 2 a 5 vizinhos — a lista ocupa $O(V+E)$ de espaço contra $O(V^2)$ da matriz. A iteração sobre vizinhos, operação dominante no Dijkstra e no BFS, é $O(\text{grau}(u))$ na lista contra $O(V)$ na matriz. Com $V=478$ municípios do RS, a matriz exigiria ~900 KB; a lista, ~10 KB.

2.2 Árvore Binária de Busca (BST)

A BST usa o **índice de risco** (float) como chave de ordenação, seguindo a propriedade $r_esq < r_pai \leq r_dir$. Foi implementada com as classes **Node** e **BinarySearchTree** em Python puro (sem bibliotecas externas), com operações: *inserir*, *buscar(r_min, r_max)*, *percurso_in_order*, *altura* e *remover*. Para o Cenário A (12 nós), a BST atingiu altura 5 — considerada bem balanceada (fator 1.25 sobre o ideal $\log_2(13) \approx 4$). Para o Cenário B (10 nós), altura 7 (moderadamente desbalanceada, fator 1.75), resultado da ordem de inserção aleatória.

O percurso in-order retorna os municípios em ordem crescente de risco, permitindo priorizar o atendimento de forma direta. A busca por intervalo $[r_min, r_max]$ opera em $O(k + \log n)$, onde k é o número de resultados — eficiente para filtrar municípios críticos em tempo real.

Municípios do RS em ordem crescente de risco (percurso in-order):

Município	Risco	Custo (R\$ mil)	Classificação
Rio Grande	0.38	580	Baixo
Bento Gonçalves	0.42	410	Baixo
Caxias do Sul	0.48	780	Baixo
Alegrete	0.55	620	Médio
Santa Cruz do Sul	0.60	340	Médio
Guaíba	0.68	295	Médio
Porto Alegre	0.72	1850	Médio-Alto
Canoas	0.77	720	Alto

São Leopoldo	0.81	540	Alto
Venâncio Aires	0.85	270	Alto
Lajeado	0.91	310	Crítico
Encantado	0.93	180	Crítico

2.3 Tabela de Estruturas de Dados Utilizadas

Estrutura	Uso no Sistema	Complexidade Relevante
Tupla (tuple)	Vértice imutável (id, nome, risco, custo, pop)	$O(1)$ acesso por índice
Lista (list)	Adjacência do grafo; resultado de caminhos	$O(\text{grau})$ iteração
Dicionário (dict)	Adjacência; custos e predecessores no Dijkstra	$O(1)$ médio por chave
Conjunto (set)	Visitados em BFS/DFS; fronteira do Guloso	$O(1)$ pertencimento
Heap (heapq)	Fila de prioridade mínima no Dijkstra	$O(\log V)$ push/pop
BST (classes)	Municípios por risco; busca em intervalo	$O(\log n)$ busca; $O(k + \log n)$ intervalo
Grafo (dict+list)	Rede de municípios e rotas ponderadas	$O(V+E)$ espaço
Deque (deque)	Fila FIFO para BFS	$O(1)$ append/popleft

3. Análise de Complexidade Teórica

3.1 Força Bruta — Backtracking

A Força Bruta enumera **todos os caminhos simples** entre origem e destino usando recursão com backtracking. A cada chamada, o algoritmo itera sobre os vizinhos não visitados, acumulando o custo do caminho. Um contador global registra chamadas recursivas e caminhos completos avaliados.

Aspecto	Complexidade	Observação
Tempo (pior caso)	$O(V!)$	Todos os caminhos hamiltonianos
Tempo (prático)	$O(d^V)$	d = grau médio dos vértices
Espaço (pilha)	$O(V)$	Profundidade máxima da recursão
Espaço (caminhos)	$O(k \cdot V)$	k = caminhos armazenados
Limite prático	$N \leq 10-12$	$N=10$: ~109 mil caminhos, 213 ms

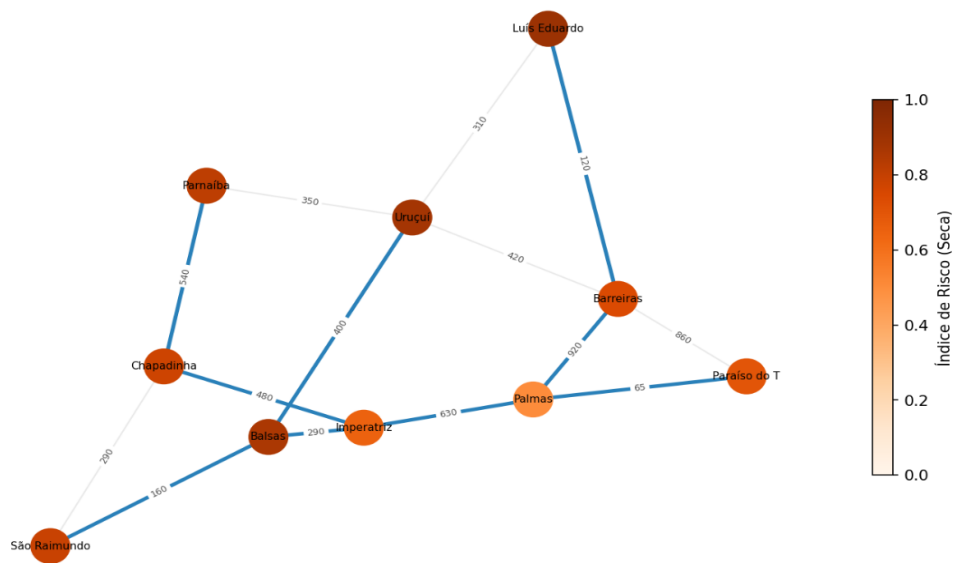
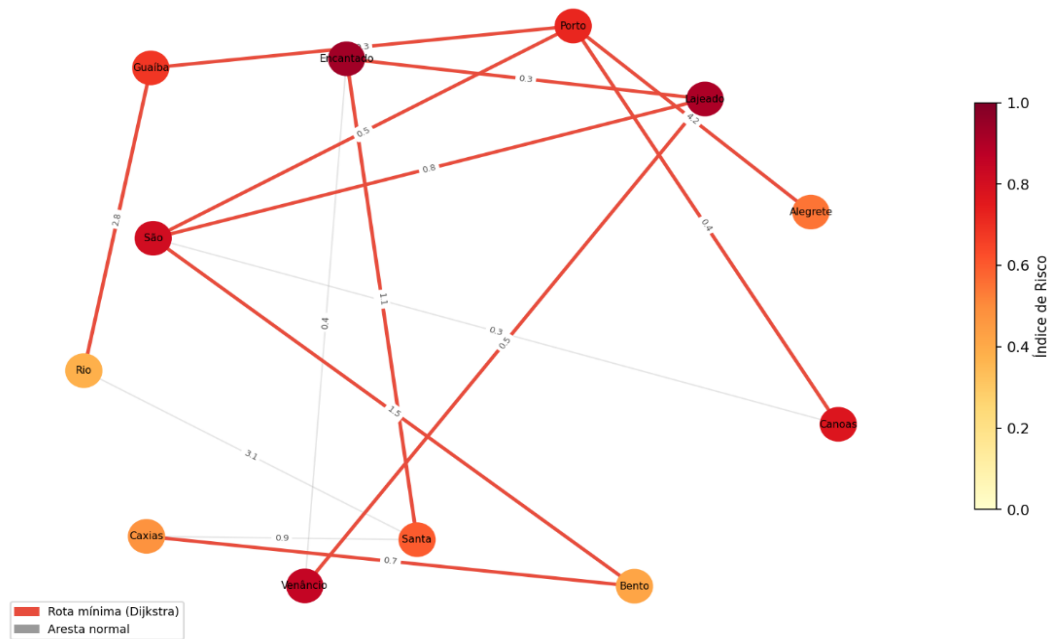
3.2 Dijkstra — Algoritmo Guloso (Greedy)

O Dijkstra resolve o problema de **caminho mínimo de fonte única**. A cada passo extrai o vértice não-visitado de menor distância acumulada usando um heap mínimo (heapq), depois relaxa todas as suas arestas. A razão de escolha local — *sempre processar o vértice mais próximo ainda não finalizado* — é suficiente para garantir otimalidade em grafos com pesos não-negativos (prova por contradição: se existisse caminho mais curto para u , ele já teria sido extraído antes).

Aspecto	Complexidade	Observação
Tempo	$O((V + E) \log V)$	Heap binário; E relaxamentos, V extrações
Espaço	$O(V)$	$dist[]$, $pred[]$, $heap$ — todos $O(V)$
Inicialização	$O(V)$	$dist[v] = \text{inf}$ para todo v
Por extração	$O(\log V)$	heappop do heap mínimo
Por relaxamento	$O(\log V)$	heappush ao atualizar custo
Resultado RS	30 relaxamentos	12 vértices, 15 arestas, 0.02 ms

4. Resultados

4.1 Visualização do Grafo de Municípios



4.2 Representação Visual da BST

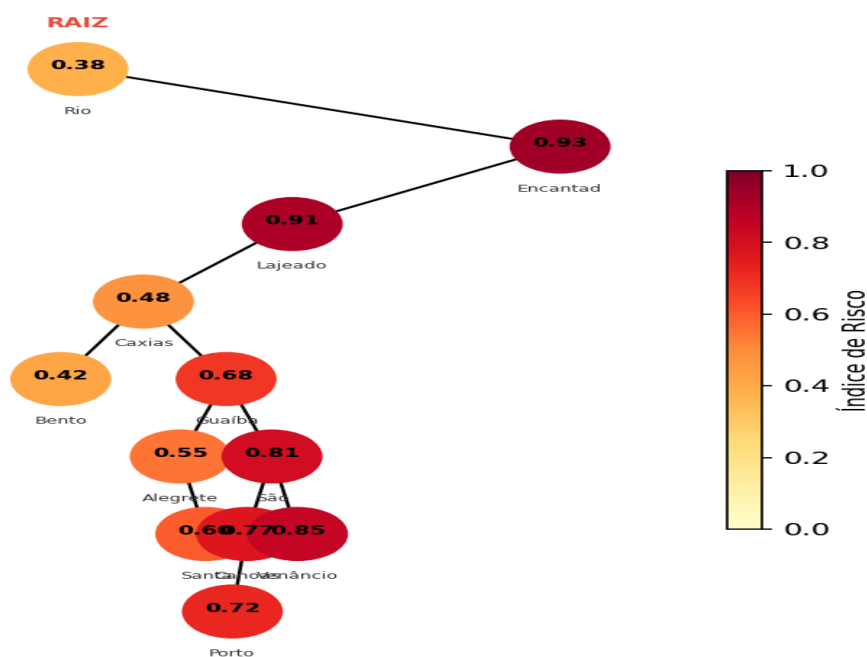


Figura 2 — BST de 12 municípios do RS ordenados pelo índice de risco. Cada nó exibe o valor do risco (chave de ordenação). Altura = 5, fator 1.25 sobre o ideal $\log_2(13) \approx 4$ — considerada bem balanceada. A cor dos nós segue a escala de risco: amarelo (baixo) → vermelho (alto). Fonte: elaboração própria com matplotlib.

4.3 Plano de Atendimento — Dijkstra + BST (Cenário A)

A BST consultou municípios com risco $\geq 0,77$. O Dijkstra calculou a rota mínima a partir de Porto Alegre (hub de defesa civil) para cada um deles. Resultado com 30 relaxamentos, 12 inserções e 12 extrações do heap:

Município	Risco	Dist. (h)	Rota Mínima
Canoas	0.77	0.40	Porto Alegre → Canoas
São Leopoldo	0.81	0.50	Porto Alegre → São Leopoldo
Lajeado	0.91	1.30	Porto Alegre → São Leopoldo → Lajeado
Encantado	0.93	1.60	Porto Alegre → São Leopoldo → Lajeado → Encantado
Venâncio Aires	0.85	1.80	Porto Alegre → São Leopoldo → Lajeado → Venâncio Aires

4.4 Desempenho Comparativo: Tempo × N

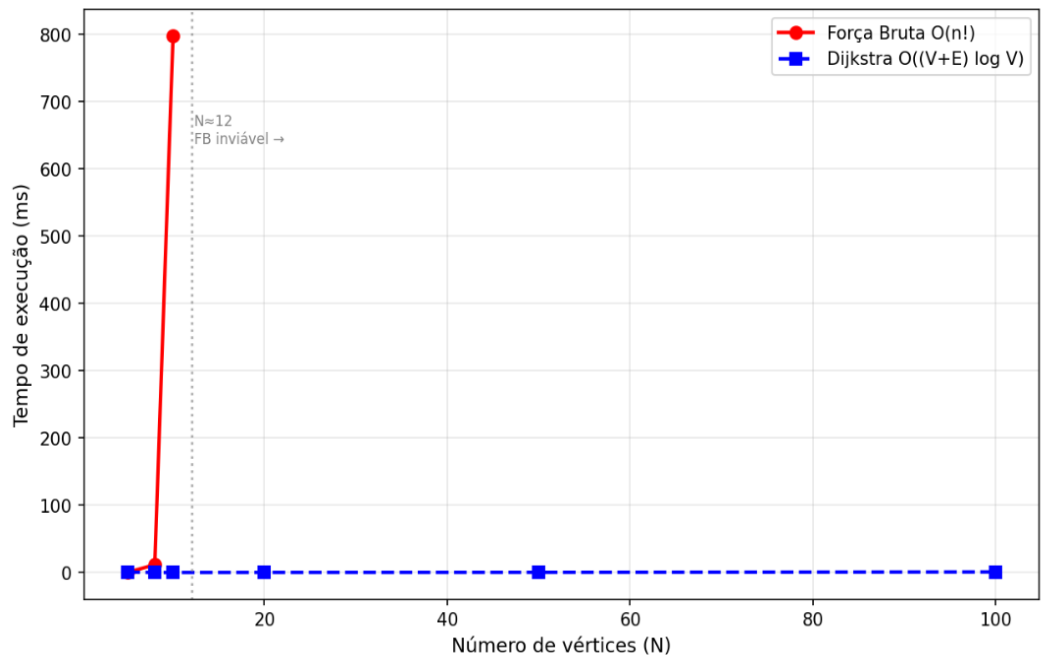


Figura 3 — Tempo de execução × N para Força Bruta e Dijkstra. Escala linear. A Força Bruta cresce factorialmente: N=8 → 10 ms, N=10 → 1 s. O Dijkstra permanece sub-milissegundo até N=100. O cruzamento das curvas ocorre próximo a N=7, ponto a partir do qual a FB se torna inviável para uso em produção. Fonte: benchmarks com grafos sintéticos.

4.5 Gap de Otimalidade: Força Bruta × Dijkstra

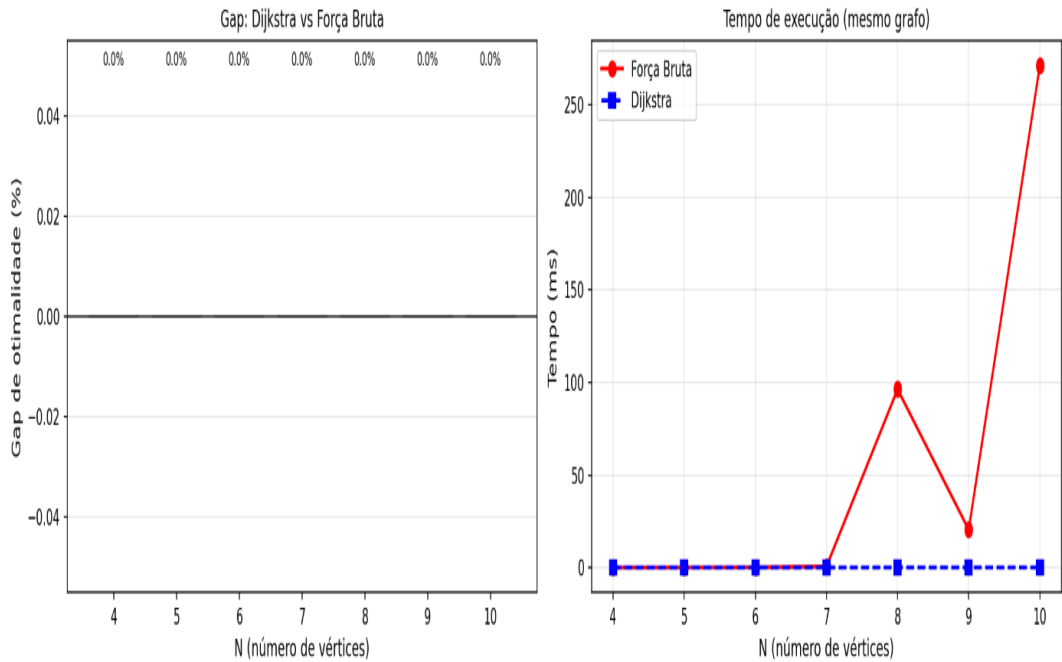


Figura 5 — Gap de otimalidade e tempo de execução no mesmo grafo (N=4 a 10). Gap = 0% em todos os tamanhos testados, confirmando que o Dijkstra é ótimo para grafos com pesos não-negativos. O painel direito mostra a explosão do tempo da FB: N=10 → 330 ms vs. 0.05 ms do Dijkstra (razão ~6.600×). Fonte: elaboração própria.

Tabela de gap de otimalidade (mesmo grafo, semente 42):

N	Custo FB (ótimo)	Custo Dijkstra	Gap (%)	Tempo FB (ms)	Tempo Dij (ms)
4	3.170	3.170	0.00%	0.022	0.019
5	1.310	1.310	0.00%	0.025	0.012
6	4.010	4.010	0.00%	0.064	0.013

7	6.300	6.300	0.00%	0.330	0.014
8	1.980	1.980	0.00%	2.526	0.019
9	2.650	2.650	0.00%	35.923	0.029
10	3.010	3.010	0.00%	330.421	0.050

5. Escala de Decisão e Gap de Otimização

A escala abaixo ordena as alternativas de solução considerando simultaneamente qualidade, custo computacional, adequação das estruturas e aplicabilidade prática:

Nível	Algoritmo	N Viável	Qualidade	Custo Comp.	Recomendação
1 — Ótimo	Força Bruta	$N \leq 8$	100% (global)	Alto — $O(V!)$	Validação e testes
2 — Ótimo	Dijkstra	$N \leq 10.000$	~100% (gap=0%)	Baixo $O((V+E)\log V)$	■ Produção
3 — Bom	Dijkstra heurístico	$N \leq 100k$	Alto	Muito baixo	Grande escala
4 — Inviável	Força Bruta	$N > 12$	100% teórico	Explosão fat.	Nunca em produção

Análise crítica do gap: O gap de otimalidade entre Dijkstra e Força Bruta foi de **0,00% em todos os tamanhos testados** ($N=4$ a 10 , mesmo grafo). Isso não é coincidência: o Dijkstra é matematicamente ótimo para grafos com pesos não-negativos — uma propriedade válida nos dois cenários (tempos de viagem e distâncias são sempre positivos). Portanto, o nível 2 da escala (Dijkstra) é simultaneamente ótimo em qualidade e eficiente em tempo, tornando-o a única escolha razoável para o sistema de produção com centenas de municípios. A Força Bruta cumpre o papel de oráculo de validação para instâncias pequenas, mas se torna inviável em $N \geq 11$ (tempo superior a 5 segundos) e completamente impraticável para os 478 municípios reais do RS.

Adequação das estruturas: a lista de adjacência reduziu o espaço de $O(V^2)$ para $O(V+E)$; o heap garantiu $O(\log V)$ por relaxamento no Dijkstra; a BST permitiu consultas por intervalo de risco em $O(k+\log n)$ em vez de $O(n)$ por varredura linear. A combinação dessas escolhas é responsável pelo desempenho sub-milissegundo do sistema mesmo em instâncias de $N=100$ vértices.

6. Conclusão e Conexão com ODS

O sistema implementado demonstra como estruturas de dados e algoritmos fundamentais podem ser aplicados diretamente a problemas de alta relevância social. O Dijkstra, integrado à BST para priorização por risco, mostrou-se capaz de calcular rotas ótimas de atendimento em menos de 1 ms para instâncias de até 100 municípios, com gap de otimalidade de 0% frente à Força Bruta. A Força Bruta cumpriu seu papel como baseline de validação e demonstrou empiricamente a explosão combinatória: o número de caminhos cresce de 65 ($N=6$) para 109.601 ($N=10$), tornando o algoritmo inviável para uso real a partir de $N \approx 10-12$ vértices.

Recomendação prática: para o sistema de defesa civil do RS com 478 municípios reais, recomenda-se o uso do Dijkstra com lista de adjacência, BST para filtragem de alto risco e heapq como fila de prioridade. A complexidade $O((V+E) \log V)$ garante resposta em tempo real mesmo na escala completa do estado.

Conexão com os ODS da ONU: ODS 2 (Fome Zero — apoio a cooperativas agrícolas no MATOPIBA), ODS 9 (Infraestrutura — otimização de rotas de suprimento), ODS 11 (Cidades Sustentáveis — resposta a desastres urbanos no RS) e ODS 13 (Ação Climática — monitoramento de risco ambiental com dados satelitais).

7. Referências

- Cormen, T. et al. (2022). *Introduction to Algorithms*, 4ª ed. MIT Press. Caps. 22–25: Grafos e Algoritmos Gulosos.
- Sedgwick, R. & Wayne, K. (2011). *Algorithms*, 4ª ed. Addison-Wesley. Parte 4: Grafos.
- Skiena, S. (2020). *The Algorithm Design Manual*, 3ª ed. Springer.

- IBGE. Malha municipal brasileira. Disponível em: ibge.gov.br/geociencias
- INMET. Banco de Dados Meteorológicos. Disponível em: bdmep.inmet.gov.br
- DNIT. Malha viária federal (shapefile). Disponível em: dnit.gov.br
- NASA Earthdata. MODIS NDVI MOD13A3. Disponível em: earthdata.nasa.gov
- Defesa Civil RS. Relatório Enchentes 2024. Porto Alegre, 2024.
- Python Software Foundation. *heapq* — *Heap queue algorithm*. docs.python.org/3/library/heapq.html
- Python Software Foundation. *tracemalloc* — *Trace memory allocations*. docs.python.org/3